

Metaprograms and fragments are needed in comma-separated contexts

Document #: P2353R0

Date: 2021-03-28

Audience: SG7

Reply-to: David Rector davrec@gmail.com

Abstract

P0712/P2237 define "metaprograms" (`constexpr { }`), within which code generation via "injection" (`<<`) of semantically structured "fragments" is permitted, with the benefit of typical function-scope semantics: control flow statements, local variables, etc.

Metaprograms are allowed in class, namespace, and block scopes. But this is a subset of all code generation contexts which require function-scope semantics, demonstrated by the fact that a cruder "string-injection" approach would permit metaprogramming that is presently impossible with P2237 facilities.

As remedy, this paper proposes:

1. Allowing metaprograms in comma-separated contexts as well: enumerator lists, (template) parameter lists, (template) argument lists, (constructor) initializer lists, and base specifier lists; and
2. Adding corresponding fragment kinds to allow injection of user-defined entities in these contexts.

In addition, a generalized fragment syntax of the form `^<K>{ }` is proposed to handle the new fragment kinds.

These changes should extend the capabilities of P2237's fragment-injection facilities to nearly match those of a string-injection approach, while avoiding its drawbacks.

Revision history

Unpublished draft -> R0:

A non-numbered draft of this paper, shared on the SG7 mailing list and entitled "The syntactic gap: string-injection metaprogramming capabilities and how the semantic approach can match them," proposed class fragment syntax equivalent to that used to reflect an anonymous class: e.g. `^class { }`. This was discarded in favor of `^<class> { }`, owing to an important distinction between reflections and fragments, discussed in section 6.

Contents

1. String vs. token-sequence vs. fragment injection
2. Under P2237, control flow is either not possible or not efficient in many injection contexts.
3. `Sum<T, U> / PreCalcdSum<T, U>` demonstrate the need for additional injection contexts with control flow
4. String-injection permits arbitrary injection with control flow, but with costs.
5. Proposal: `constexpr { }` should be permitted in "comma-separated" contexts, and new fragment kinds introduced.

6. Proposal: new fragment syntax `^<K>{ }`
7. `PreCalcdSum<T, U>` implementation
8. Alternative: replace `constexpr { }` with `<< or ... [ :: ] ...`
9. Packs: syntactic sugar for fragment-injecting metaprograms?

1. String vs. token-sequence vs. fragment injection

P2237 [1] summarizes metaprogramming facilities under advanced consideration, among them the "metaprogram" `constexpr { }` (originally introduced in P0712 as a "constexpr block" [2]) and the injection operator `<<`.

What kinds of things should follow `<<`? P0633 [4] first delineated the three rough possibilities, which today might be distinguished as follows: 1) strings, 2) token-sequences, and 3) fragments.

1.1 String injection.

In a string injection approach, the user constructs, manipulates, and injects strings containing ordinary code syntax at compile time; the compiler parses the resulting strings as dependencies are sufficiently resolved. Such an approach is used by D string mixins [3](#), but is also possible in C++ [4, 5]. An example of possible C++ syntax:

```
template<struct T>
class Interface {
public:
    constexpr {
        template for (auto data : meta::data_members_of(^T))
            << format("virtual {} {}() const = 0;",
                    meta::qualified_name_of(meta::type_of(data)),
                    "get_" + meta::name_of(data));
    }
};
constexpr void inject_field_or_var() {
    << "int i = 4;"
}
class A {
    // Injects a field named `i`, with default initializer 4.
    constexpr { inject_field_or_var(); }
};
void f() {
    // Injects a variable declaration `i` initialized to 4.
    constexpr { inject_field_or_var(); }
}
```

1.2 Token-sequence injection

Token-sequence injection improves upon the readability of string injection by keeping the user's injected content out of quotation marks, but is little different semantically, storing dependent parsed content in an unstructured manner (as a sequence of tokens). Some string manipulation and injection is still possible, but is restricted to producing identifier tokens. Possible syntax of token-sequence injection, adopting the `[ :: ]` and `|##|` identifier splice syntax from P2237/P2320:

```
template<struct T>
class Interface {
public:
    constexpr {
        template for (auto data : meta::data_members_of(^T))
            << { virtual [ :meta::type_of(data): ]
                |#"get_" + meta::name_of(data)|##|() const = 0; };
    }
}
```

```

};
consteval void inject_field_or_var() {
    << { int i = 4; };
}
class A {
    // Injects a field named `i`, with default initializer 4.
    consteval { inject_field_or_var(); }
};
void f() {
    // Injects a variable declaration `i` initialized to 4.
    consteval { inject_field_or_var(); }
}

```

1.3 Fragment injection

Fragment-injection only permits injection of entities whose semantics are unambiguous [11, 12]. This approach is accomplished by simply requiring the user to specify the "kind" of the entities to be injected; otherwise the syntax is identical to token-sequence injection. An example of the P2237 syntax:

```

template<struct T>
class Interface {
public:
    consteval {
        template for (auto data : meta::data_members_of(^T))
            << <class { // a "class fragment"
                virtual typename [:meta::type_of(data):]
                    |#"get_" + meta::name_of(data)#|() const = 0;
            }>;

        // (NOTE: We omit use of the "unquote operator" %{} [12, 1])
        // here and in all other fragment examples, for simplicity.)
    }
};
consteval void inject_field() {
    << <class { int i = 4; }>; //class fragment injection
}
consteval void inject_vardecl() {
    << <{ int i = 4; }>; //statement injection
}
class A {
    // Injects a field named `i`, with default initializer 4.
    consteval {
        //inject_vardecl(); //ERROR
        inject_field();
    }
};
void f() {
    // Injects a variable declaration `i` initialized to 4.
    consteval {
        //inject_field(); //ERROR
        inject_vardecl();
    }
}

```

These approaches produce the same instantiated declarations in any target program. Which is best?

The benefit of string-injection is that its capabilities are limitless: users can generate any entity whose syntax they can represent via compile time string manipulations, typically involving many string-returning reflection queries.

The downsides of string-injection have been discussed extensively in P0633 and on the SG7 mailing list, and include such matters as poor syntax highlighting, a tendency to produce "write-only" code, and difficult

debugging and semantic analysis. [4, 6, 7]

The token-sequence injection approach improves readability and syntax highlighting abilities, but semantic analysis remains nearly as problematic as under the string-injection approach, due to the similar unstructured nature of the content. Worse still, the token approach loses the arbitrary capabilities of the string-injection approach (as generally suggested in [4] but more specifically evident from the examples considered below, which are just as impossible for token-injection as they are for fragment-injection).

Fragment-injection improves further upon the token approach by allowing the compiler to verify as much information as possible about the injected entities' syntax and semantics, even in the presence of dependencies (i.e. in templates, prior to instantiation). To be sure: because identifiers may still be generated from string manipulations, some semantic verification must wait until instantiation even under this approach, but this problem seems unavoidable in any sufficiently capable metaprogramming facilities.

The fragment approach seems best *so long as* it is as capable as a string-injection approach. To the extent it is not as capable, the choice is less clear.

The mechanisms summarized in P2237 are not yet as capable as a string-injection approach, but can be.

2. Under P2237, control flow is either not possible or not efficient in many injection contexts.

Section 3 of P0712 describes the goal of metaprograms ("constexpr blocks" then) and the fragment-injection facilities therein as to provide "the ability to execute constexpr in (nearly) any context, and the ability to specify what code gets injected." [2]

However, metaprograms are presently only permitted in class, namespace, and block scopes, which is a subset of all injection contexts. Injection *with control flow* is a) not possible -- or at least not efficient -- into expression lists, enumerator lists, and parameter lists; and b) is certainly not possible in still other contexts for which the necessary fragment kinds do not yet exist.

Regarding (a): reproduced below are excerpts from relevant sections of P2237, providing syntax examples of the unusual injection mechanisms permitted in expression, enumerator, and parameter contexts:

P2237 Section 7.1.5

...Expression fragments are typically inserted into a program via splicing. Also, if the expression fragment contains multiple initializers, then the splice must be expanded.

```
constexpr meta::info inits = <(1, 2, 3)>;  
vector<int> vec{ ...[:inits:]... }; // (adjusted to use P2320 [8] syntax)
```

The resulting vector is initialized with the values 1, 2, and 3.

P2237 Section 7.2.2

Enumeration fragments can be injected into enums using an injection enumerator:

```
constexpr meta::info rgb = <enum { red, blue, green }>;  
constexpr meta::info cmy = <enum { cyan, magenta, yellow }>;  
enum class color {  
    << rgb;  
    << cmy;  
};
```

P2237 Section 7.2.6

```
int f(auto... params << meta::parameters_of(some_fn)) {
    eat(params...);
}
```

While each of these unusual, outside-of-constexpr {} injection mechanisms may be defensible, by living outside constexpr {} they offer no apparent, efficient mechanism to add control flow to specify precisely which entities should be injected, and which should not. (To be sure, a combination of P2237 facilities *might* allow an *inefficient* means of injecting with control flow into these contexts; this possibility, and its fatal inefficiency, is discussed in section 8.)

In still other contexts -- for example, constructor initializer lists -- injection is not possible at all (even inefficiently); that is, in such contexts there is not even the possibility of using unusual ... [::] ... or << mechanisms to inject user-defined entities, because the necessary fragment kinds do not exist.

Is this really a problem? Is injection with control flow really needed in these contexts? The following class of examples suggest it is.

3. Sum<T, U> / PreCalcdSum<T, U> demonstrate the need for additional injection contexts with control flow

Sum<T, U> and PreCalcdSum<T, U> are examples whose implementation requires additional injection contexts with control flow. They were roughly described on the SG7 mailing list as follows: [13](#)

```
template<class T, class U>
class Sum {
    T t;
    U u;
public:
    constexpr Sum(T &t, U u) : t(t), u(u) {}

    // Methods: the union of the methods of T and U.
    // Wherever they "share" a method, such that names and
    // signatures of method reflections m_t and m_u are the same
    // (not necc. via inheritance) that method is implemented
    // to return t.[:m_t:](...) + u.[:m_u:](...). Otherwise, it returns
    // t.[:m_t:](...) or u.[:m_t:](...) individually.

    // Conversion operators: construct from the relevant fields,
    // but for any data T shares with U, need to add in the other's
    // data to the initializer.
    constexpr explicit operator T();
    constexpr explicit operator U();
};

template<class T, class U>
class PreCalcdSum {
    // Data members: union of the "unique" data members
    // of T and U.

public:
    // Initializes each of the data members above, again as either sum
    // from t and u or results from t or u individually
    constexpr PreCalcdSum(T t, U u);

    // Methods: same as before, union of methods of T and U,
    // but now implemented in terms of above-calculated data.
```

```
// Conversion operators: construct from the relevant fields
constexpr explicit operator T();
constexpr explicit operator U();
};
```

On the SG7 list Ville Voutilainen notes the broader utility of further generalizations of `Sum<T,U>`:

Such things certainly should be supported. It's just another variant of a generic Mediator that invokes multiple Observers, and summing is just one of many things that one might want to do with such a thing, others include, for example, collecting the results in a tuple, and collecting the results into a vector. The generic Mediator has the same functions its Observers have, and it just injects those functions into its own interface by reflecting the interface(s) of the Observer(s). We write these things fairly often, and reflection+injection makes it significantly less boilerplatey and more generic. And *using* such a generic Mediator makes it very easy to write such combinations without the status quo of excessive boiler-plate programming. [9]

Neither `Sum<T,U>` nor `PreCalcdSum<T,U>` can be implemented with existing P2237 facilities. They can be implemented with string injection, however.

4. String-injection permits arbitrary injection with control flow, but with costs.

String-injection allows for arbitrary code generation with arbitrary control flow in the above contexts and any other conceivable contexts, such as constructor initializers:

```
template<struct T, struct U>
class PreCalcdSum {
    ...
private:
    static constexpr void colon_or_comma(bool &first) {
        if (first) {
            << ":";
            first = false;
        } else
            << ",";
    }
    static constexpr set<const char *> Tset = ...; // members of T
    static constexpr set<const char *> Uset = ...; // members of U
    static constexpr set<const char *> TUset = ...; // members of TU
public:
    constexpr {
        << "constexpr PreCalcdSum(T t, U u)";
        bool first = true;
        for (auto name : TUset) {
            colon_or_comma(first);
            << name << "(";
            bool inT = Tset.contains(name);
            bool inU = Uset.contains(name);
            if (inT) {
                if (inU)
                    << format("t.{} + u.{}", name, name);
                else
                    << format("t.{}", name);
            } else
                << format("u.{}", name);
            << ")";
        }
        << "{}";
    }
};
```

This IO-style is perfectly capable of performing any injection task with arbitrary control flow, but the drawbacks discussed earlier are apparent: this code is difficult to read, unpleasant to write, and due to its unstructured nature hides information from the template about entities which occur in each instantiation, which inhibits semantic analysis and can obscure bugs which only present during certain instantiations. Essentially: unstructured injection is to structured/"fragment" injection as macros are to templates.

Nevertheless, the gap in capabilities remains: P2237 facilities cannot write the `constexpr PreCalcdSum(T t, U u)` constructor at all.

Fortunately, it is straightforward to extend the P2237 facilities to match string injection capabilities, and thereby allow fragment-injection to subsume more (all?) of the capabilities of string-injection, while avoiding its drawbacks.

5. Proposal: `constexpr {}` should be permitted in "comma-separated" contexts, and new fragment kinds introduced.

`constexpr {}` is presently allowed in class, namespace, and block scopes. It should additionally be allowed in: a) the expression, enumerator, and function injection contexts referenced above, and b) other "comma-separated" contexts which require new fragment kinds as well.

To state the goal more generally: in order to match string-injection capabilities, it seems *any* context in which a variable number of entities can be supplied (declarations, statements, expressions, initializers, template arguments, etc.) should be considered an "injection context", with an associated fragment kind and the ability to accept one or more `constexpr {}` metaprograms amidst its entities.

5.1 Fragment injection with control flow in various contexts

```
constexpr meta::info inits = <(1, 2, 3)>;
vector<int> vec{
    constexpr {
        template for (auto init : inits) {
            if (somecond(init))
                << init;
        }
    },
    4, 5, 6
};
...
enum class color {
    constexpr {
        if (cond)
            << rgb;
        else
            << cmy;
    },
    grue
};
...
int f(int i,
    constexpr {
        template for (auto p : meta::parameters_of(some_fn))
            if (cond(p))
                << p;
    },
    int j)
{
    eat(i,
        constexpr {
```

```

        template for (auto p : meta::parameters_of(some_fn))
            if (cond(p))
                << <( [:p:] ) >;
    },
    j));
}

template<consteval { /*temp param fragment injections*/ },
        typename T>
class SomeTemplate;

SomeTemplate<consteval { /*temp arg fragment injections*/ }, T>;

Foo(const T& t, const U& u)
: bar(u.bar),
  consteval { /*ctor init fragment injections*/ },
  other(t.other)
{}

class B : consteval { /* base spec. fragment injections */ },
        private Foo
{};

```

6. Proposal: new fragment syntax

There five fragment kinds are introduced in P2237 section 7:

- namespace fragments: <namespace {}>
- class fragments: <class {}> / <struct {}>
- enum fragments: <enum {}>
- statement fragments: <{}>
- expression fragments <()>

This syntax is elegant as is.

However, the previous section has introduced additional needs for:

- parameter fragments
- template parameter fragments
- template argument fragments
- initializer fragments
- constructor initializer fragments
- base specifier fragments

and perhaps still others.

It is difficult to conceive of any elegant symbolic extensions to the P2237 syntax that would incorporate these new fragment kinds. Instead, a new fragment syntax is proposed which can handle an arbitrary quantity of fragment kinds.

Two categories of fragments require separate discussion:

1. Class, namespace, and enum fragments vs.
2. All others.

6.1 Proposed syntax

```

^<K>{ }

```


where κ is a non-dependent identifier or keyword indicating the fragment kind.

Statement and expression fragment kinds would be changed as follows:

```
^<frag::stmt>{ [:m:] = 42; } // was <{ [:m:] = 42; }>
^<frag::expr>{ 3, [:m:] + 4 } // was <( 3, [:m:] + 4 )>
```

Six new fragment kinds would be introduced:

```
^<frag::parm>{ int i, int j }
^<frag::tparm>{ typename T, int N }
^<fram::targ>{ [:Trefl:], [:Nrefl:] }
^<frag::init>{ 3, .[:m:] = 4 }
^<frag::cinit>{ [:m:](3), [:n:](4) }
^<frag::base>{ public Foo, private virtual [:Barrefl:] }
```

Each above identifier κ in $^<\kappa>\{\}$ could be interpreted either of two ways:

1. As a built-in value. The type of the expression would remain `meta::info`.
2. As a built-in fragment type, each a subclass of `meta::info`. The type of the expression would then be κ .

Option 2 seems preferable, but option 1 is also reasonable, should implementation concerns demand all reflections and fragments be of `meta::info` type and absolutely no more.

6.2 Class, enum, and namespace fragments

Class fragments must allow for base specification (`<class : Base { }>`), and so require additional consideration.

Enum and namespace fragments do not have this problem, but it would be advisable to handle them similarly to class fragments for uniformity.

It would *seem* ideal to use this opportunity to link fragment specification syntax to reflection syntax, by expressing class, namespace, and enum fragments as reflections of anonymous entities of those kinds:

```
auto f1 = ^class : private Base { }; // was <class : private Base{ }>
auto f2 = ^namespace { };           // was <namespace { }>
auto f3 = ^enum { };                 // was <enum { }>
```

However, reflections and fragments are not quite the same, because they are injected differently: an injection of a reflection (even an anonymous one) injects that declaration, whereas injection of a fragment injects the *members* of the fragment. E.g., if the above syntax were used, this confusion might result:

```
class Foo { enum { MaskA = 0x1 } };
class Bar {
    consteval {
        // Injects anonymous enum containing MaskA
        template for (auto mem : meta::members_of(^Foo))
            << mem;

        //ERROR: cannot inject enum fragment into class
        << ^enum { MaskB = 0x2 };
    }
};
```

The better solution is to use $^<\kappa>\{\}$ syntax here as well:

```
^<class : private Base>\{\} // was <class : private Base{ }>
^<enum>\{\}                 // was <namespace { }>
```

```
^<namespace>{} // was <enum { }>
```

As before, the type of each expression could either remain `meta::info`, or become these subclasses thereof: `frag::cls`, `frag::ns`, and `frag::en` respectively.

It might also be reasonable to write these as

```
^<frag::cls : private Base> {}  
^<frag::ns> {}  
^<frag::en> {}
```

which would make class fragments the only idiosyncratic fragment kind -- and even then, only when a base specifier is needed, which is probably rare. The downside of this alternative is that it would not make full use of the existing language keywords. The optimal choice is a matter of taste.

Possible language conflicts of `^` are discussed in P2237 and do not appear to be a factor here either.

In sum, this dual fragment syntax can be extended to arbitrary new fragment kinds as future needs are identified, and links fragment syntax to reflection `^` syntax, which is suitable because reflections and fragments are precisely the two `meta::info`-typed expressions able to take non-`meta::info` operands. That is, these are the two "entry points" to higher level "meta" space, and the syntax should naturally imply this commonality.

7. `PreCalcdSum<T, U>` example

Only a simplified version of `PreCalcdSum<T, U>` will be demonstrated here, which sufficiently demonstrates the proposed syntax and capabilities.

Note again that usage of the "unquote operator" `%{ } [12, 1]` is omitted in this syntax, as it was in previous fragment examples, for simplicity.

```
// --- PreCalcdSum.h --- //

/// A "sum" of arbitrary objects of types T and U.
///
/// That is: each instantiation's fields are a (set-)union
/// of the fields of T and U.
/// For any fields of unqualified name `fieldname` which
/// T and U share (not necc. via inheritance), that field
/// is constructed with the value
/// t.|#fieldname#| + u.|#fieldname#|.
/// Otherwise, that field is constructed with the value
/// t.|#fieldname#| or u.|#fieldname#| alone.
///
/// For now we assume T and U each have public data and no
/// named methods. Each instantiation of PreCalcdSum<T,U>
/// also has these properties.
///
template<struct T, struct U>
class PreCalcdSum {

    // --- Compile-time data calculation
    //      (no new syntax here) --- //
    using comptime_data_t =
        tuple<set<string>,
            set<string>,
            set<string>,
            map<string, meta::info>>;
    static constexpr comptime_data_t comptime_data = [](){
        comptime_data_t data;
        auto &Tset = std::get<0>(data);
```

```

auto &Uset = std::get<1>(data);
auto &TUset = std::get<2>(data);
auto &types = std::get<3>(data);

template for (tfield : meta::fields_of(^T)) {
    string name = meta::name_of(tfield);
    Tset.insert(name);
    TUset.insert(name);
    types[name] = meta::type_of(tfield);
}
template for (ufield : meta::fields_of(^U)) {
    string name = meta::name_of(ufield);
    Uset.insert(name);
    bool inTtoo = TUset.insert(name).second;
    if (inTtoo) {
        meta::info &type = types[name];
        type = ^decltype(
            std::declval<[:type:]/*tfield type*/>() +
            std::declval<decltype([:ufield:])>() );
    } else
        types[name] = meta::type_of(ufield);
}
return data;
})();
static constexpr auto &Tset = std::get<0>(comptime_data);
static constexpr auto &Uset = std::get<1>(comptime_data);
static constexpr auto &TUset = std::get<2>(comptime_data);
static constexpr auto &types = std::get<3>(comptime_data);

// --- Class fragment injection --- //
public:
    consteval {
        for (auto name : TUset)
            << ^<struct>{ typename [:types[name]:] |#name#|; };
    }

private:
    static consteval void
    inject_cinits_from_tu(meta::info trefl, meta::info urefl) {
        for (auto name : TUset) {
            if (Tset.contains(name)) {
                if (Uset.contains(name))
                    << ^<frag::cinit>{ |#name#|([:trefl:].|#name#| +
                                                    [:urefl:].|#name#|) };
                else
                    << ^<frag::cinit>{ |#name#|([:trefl:].|#name#|) };
            } else
                << ^<frag::cinit>{ |#name#|([:urefl:].|#name#|) };
        }
    }

public:
    constexpr PreCalcdSum(const T &t, const U &u)
        : consteval { inject_cinits_from_tu(^t, ^u); }
    {}

// --- Parameter and constructor-initializer injection --- //
private:
    static consteval void
    inject_fields_as_parms() {
        for (auto name : TUset)
            << ^<frag::parm>{ [:types[name]:] |#name#| };
    }
    static consteval void

```

```

inject_fields_as_cinits() {
    for (auto name : TUset)
        << ^<frag::cinit>{ |#name#|(|#name#|) };
}
public:
constexpr PreCalcdSum(consteval { inject_fields_as_parms() })
    : consteval { inject_fields_as_cinits() }
{}

// --- Initializer fragment injection --- //
constexpr operator T() const {
    return T{
        consteval {
            for (auto name : Tset)
                << ^<frag::init>{ .|#name#| = |#name#| };
        }
    };
}
constexpr operator U() const {
    return U{
        consteval {
            for (auto name : Uset)
                << ^<frag::init>{ .|#name#| = |#name#| };
        }
    };
}
};

```

A test usage of PreCalcdSum:

```

#include "PreCalcdSum.h"

template <class T, class U>
class operator_plus_defined {...}; //sfinae detector

template<typename T, typename U,
        typename = std::enable_if_t<!operator_plus_defined<T, U>::value>>
PreCalcdSum<T, U> operator+(T t, U u) { return PreCalcdSum<T, U>(t, u); }

struct A {
    int f;
    int g;
};
struct B {
    int g;
    A h;
};
struct C {
    int f;
    int g;
    B h; //NB different type from B::h
};

int main() {
    A a1{.f = 1, .g = 2};
    A a2{.f = 3, .g = 6};
    B b{.g = 3, .h = A{.f = 4, .g = 5}};
    C c{.f = 7, .g = 1, .h = B{.g = 3, .h = A{.f = 1, .g = 2}}};

    auto sum = a1 + a2 + b + c;

    assert(sum.f == 11);
    assert(sum.g == 12);
    assert(sum.h.g == 8);
}

```

```

    assert(sum.h.h.f == 1);
}

```

8. Alternative: replace `consteval {}` with `<< or ...[::]...`

The new fragment kinds discussed above seem to be necessary to advance the goal of matching string-injection capabilities.

However, there is a reasonable alternative to *how* fragments should be injected, based on certain unusual injection facilities already present in P2237.

Section 7.2.3 of P2237 describes the injection operator `<<` as able to inject into an alternative context; e.g. one can write `dst << frag` to inject `frag` into `dst` (which must be another `meta::info` object).

This permits treating a fragment like a container, which in turn permits an intriguing possibility: `consteval {}` can technically be done away with altogether, at least as a mechanism for injecting with control flow, and replaced with either `...[::]...` or `<<` syntax everywhere injection with control flow is needed.

Here is how the relevant parts of the `PreCalcdSum<T,U>` example might be written using `...[::]...` syntax exclusively in place of `consteval {}`:

```

public:
    // --- Class fragment injection --- //
    ...[::]
    []() {
        auto res = ^<struct> {};
        for (auto name : TUset)
            res << ^<struct> { [:types[name]:] |#name#|; };
        return res;
    }()
    :]...;

    // --- Constructor initializer injection --- //
private:
    static consteval meta::info
    build_cinits_from_tu(meta::info trefl, meta::info urefl) {
        auto res = ^<frag::cinit>{};
        for (auto name : TUset) {
            if (Tset.contains(name)) {
                if (Uset.contains(name))
                    res << ^<frag::cinit>{ |#name#|([:trefl:].|#name#| +
                                                    [:urefl:].|#name#|) };
                else
                    res << ^<frag::cinit>{ |#name#|([:trefl:].|#name#|) };
            } else
                res << ^<frag::cinit>{ |#name#|([:urefl:].|#name#|) };
        }
        return res;
    }
public:
    constexpr PreCalcdSum(const T &t, const U &u)
        : ...[:build_cinits_from_tu(^t, ^u):]...
    {}

    // --- Parameter and constructor-initializer injection --- //
private:
    static consteval auto build_fields_as_parms() {
        //...
    }
    static consteval auto build_cinits_from_parms() {
        //...
    }

```

```

    }
public:
    constexpr PreCalcdSum( ...[:build_fields_as_parms():]... )
        : ...[:build_cinits_from_params():]...
    {}

    // Initializer injection
private:
    static consteval auto
    build_inits_for_set(const decltype(Tset) &nameset) {
        //...
    }

public:
    constexpr T operator T() {
        return T{ ...[:build_inits_for_set(Tset):]... };
    }
    constexpr U operator U() {
        return U{ ...[:build_inits_for_set(Uset):]... };
    }
};

```

This alternative offers the exact same capabilities to `constexpr {}`, and the `...[::]...` syntax might be considered preferable. Which is better?

The `constexpr {}` version seems to win handily on build time-efficiency grounds. Whereas `constexpr { inject_stuff() }` can inject directly into the ultimate destination, `...[:buildfrag():]...` requires copying the contents of the result of `buildfrag()` into the ultimate destination during evaluation. One cannot rely on compiler heroics to save us from this, as they too count against efficiency, and therefore rarely show up for constant evaluation tasks -- see the rationale for the `meta::info` type given in P1240, which points out that that even basic subobject access is not optimized during constant evaluation in major compilers [9].

In short, it seems we really do need `constexpr {}` directly in the destination contexts for efficient injection with control flow.

9. Packs: syntactic sugar for fragment-injecting metaprograms?

This paper has proposed extending metaprograms with associated fragment kinds beyond the class, namespace, and block scopes, as P2237 presently allows for, and into "comma-separated" contexts.

To some extent, this goal is an inversion of P1858 [14], which proposes to extend pack expansions and declarations beyond comma-separated contexts and into class, namespace, and block scopes; e.g.:

```

template <typename... Ts>
struct tuple {
    Ts... elems;
};
template <typename... Ts>
void foo(Ts&&... ts) {
    using ...decayed = std::decay_t<Ts>;
    auto... copies = std::forward<Ts>(ts);
}

```

The overlap between the contexts in which pack and metaprogram usages are either presently allowed or proposed suggests a more general principle: pack expansions and declarations might best be considered syntactic sugar for fragment-injecting metaprograms, suitable for use in these circumstances:

1. in the case of pack expansions, the entities to be expanded do not require filtering or other fine-grained manipulations via control flow statements, or

2. in the case of pack declarations, the entities to be declared to not require idiosyncratic names, and can instead be referenced via e.g. `elems...[I]`.

This is often the case, but not always, as suggested by the `Sum<T,U>` and `PreCallocSum<T,U>` examples.

Thus, perhaps a more general principle can be stated: in any C++ context which allows for a variable number of entities (declarations, statements, parameters, etc.), the language should allow for each of the following:

1. Metaprograms (`constexpr {}`)
2. A fragment kind for that entity (`^<frag::someentity>{ }`)
3. Pack expansions and/or pack declarations

The relationship between metaprograms, fragments, and pack usage, and in particular how their relationship might be expressed generically across all contexts, warrants additional thought.

References

- [1]: Sutton, A. "Metaprogramming." P2237, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2237r0.pdf>, 10/2020.
- [2]: Sutton, A., Sutter, H. "Implementing language support for compile-time metaprogramming." P0712, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0712r0.pdf>, 06/2017.
- [4]: Vandevorde, D., Dionne, L. "Exploring the design space of metaprogramming and reflection." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0633r0.pdf>, 03/2017.
- [5]: Rector, D. <https://github.com/drec357/clang-meta>, 2019 (implementation of C++ string-injection).
- [6]: <https://lists.isocpp.org/sg7/2020/10/0071.php>, 10/2020.
- [7]: <https://lists.isocpp.org/sg7/2020/10/0053.php>, 10/2020.
- [8]: Sutton, A., Childers, W., Vandevorde, D. "The Syntax of Static Reflection." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2320r0.pdf>, 02/2021.
- [9]: Childers, W., Sutton, A., Vali, F., Vandevorde, D. "Scalable Reflection in C++." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1240r1.pdf>, 10/2018.
- [10]: <https://lists.isocpp.org/sg7/2021/02/0187.php>, 02/2021
- [11]: Sutton, A., Childers, W. "Compile-time Metaprogramming in C++." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1717r0.pdf>, 06/2019.
- [12]: Sutton, A., Childers, W. "Tweaks to the design of source-code fragments." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2050r0.pdf>, 01/2020.
- [14]: Revzin, Barry. "Generalized pack declaration and usage." <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1858r2.html>, 03/2020.